Algorismes Avançats

Grau en Enginyeria Informàtica

Pràctica 1: Complexitat Algorísmica

Autors:

Josep Oliver
Hugo Valls

Universitat de les Illes Balears

12 de març de 2026

Índex

1 Introducció **2**

2 Arquitectura MVC **2**
2.1 El Model 2
2.2 La Vista 2
2.3 El Controlador 3

3 Problemes del Maquinari i Solucions **3**
3.1 El fenomen del Jitter 3
3.2 L'escalfament de Java (Warm-up) 3

4 Estratègies de Renderització Eficient **4**
4.1 Optimització del Càlcul per Mostreig 4
4.2 Mapeig de Coordenades 4
4.3 Normalització Dinàmica i Escalat 4

5 Guia d'Usuari **5**
5.1 Interfície i Inici de la Simulació 5
5.2 Selecció d'Algorismes 5
5.3 Interpretació de Resultats 5

6 Conclusions **6**

Resum

Aquest document detalla el desenvolupament d'una aplicació visual per a l'avaluació de la complexitat asimptòtica d'algorismes, especialment les complexitats: ($O(n), O(n \log n), O(n^2), O(n^3)$). En aquest treball posarem un èmfasi especial en els reptes que hi ha d'avaluar costos computacionals a l'escala de nanosegons sobre un ordenador modern. Presentarem solucions avançades, com l'optimització del compilador *Just-In-Time* (JIT) de la JVM, l'ús d'una estructura de programació *Model Vista Controlador* (MVC), i l'optimització de les gràfiques que dibuixarem. Volem destacar l'ús d'un algorisme basat en percentatges relatius per al dibuix independent de la resolució i la segmentació del mostreig (calculant únicament els punts clau de la gràfica) per estalviar cicles innecessaris de CPU en progressions elevades.

1 Introducció

Saber quant tardarà un programa a fer la seva feina és una part bàsica de la informàtica. Normalment, a classe fem servir la notació Big-O per calcular això en teoria, mirant el codi i comptant els bucles que hi ha.

El problema és que la teoria a la pissarra és una cosa, i executar el codi en un ordinador de veritat n'és una altra de molt diferent. En aquesta pràctica, el nostre objectiu principal és comprovar si aquesta teoria s'ajusta a la realitat, mesurant el temps d'execució de diferents algorismes programats en *Java*.

Per fer-ho, hem creat una aplicació visual interactiva amb `Swing` seguint l'estructura Model Vista Controlador (MVC). El repte més gran que ens hem trobat ha estat fer que el programa funcioni bé i no es pengi en provar valors grans. Per exemple, si volem provar un algorisme de $O(n^3)$ amb un valor de $N = 5.000$, l'ordinador ha de fer 125.000 milions d'operacions de cop. En aquesta memòria explicarem com hem solucionat aquests problemes pràctics, com hem evitat els bloquejos de pantalla i com hem dissenyat la gràfica perquè els resultats es puguin veure de forma clara.

2 Arquitectura MVC

L'organització d'una aplicació interactiva requereix d'una divisió de "responsabilitats" per evitar l'acoblament. S'ha implementat el paradigma MVC seguint la següent organització:

2.1 El Model

Actua com la base de les operacions. Aquesta classe està totalment desconnectada de la representació visual i el control, apart d'això té dos objectius:

1. Generar els bucles de complexitat (simulació) i retornar el valor del temps en nanosegons.
2. Emmagatzemar les dades de cada complexitat.

Tots els temps calculats es transformen en instàncies d'un objecte `Punto(N, Temps)`. Aquesta classe `Punto` simplement representa el que es un punt dins la gràfica, l'eix x serà el valor de n i l'eix y serà el valor del temps que tarda l'algorisme. A més, aquests punts es guarden a llistes pròpies per cada complexitat.

2.2 La Vista

Controla la construcció dels `JFrames` i `JPanels`. S'ha separat la gestió de finestres, botons i menús de l'algorísmica del *rendering* matemàtic o sigui, de les corbes, que s'ubica exclusivament en el mètode sobreescrit `paintComponent` de `PanelGrafico`.

- **`Vista.java`:** És la finestra principal (hereta de `JFrame`). S'encarrega de l'esquelet de l'aplicació: col·loca els botons d'inici i aturada, crea la barra de menú superior per triar quin algorisme volem provar i actualitza uns textos informatius (`JLabel`) on es mostren els nanosegons exactes que triga cada passa. Per respectar completament l'arquitectura MVC, aquesta classe no decideix què fan els botons quan es cliquen; només els prepara, els mostra, i els exposa mitjançant mètodes *getter* perquè el Controlador hi assigni la lògica posteriorment.

- `PanelGrafico.java`: És el nostre llenç de dibuix (hereta de `JPanel`). S'ha separat estratègicament la gestió de finestres, botons i menús de del *rendering* de les gràfiques, que s'ubica exclusivament aquí, sobreescrivint el mètode `paintComponent`. Aquesta classe fa servir la llibreria `Graphics2D` per aplicar *anti-aliasing* (perquè les línies es vegin suaus i no pixelades), dibuixa els eixos geomètrics de la pantalla i uneix els punts que li envia el Controlador per formar les corbes representant cada complexitat amb un color diferent.

2.3 El Controlador

El Controlador és la peça que fa de pont entre la Vista i el Model. Aquí és on hem programat com respòn el programa quan l'usuari interactua amb la interfície. Hem fet servir les interfícies `ActionListener` per "escoltar" els clics dels botons i del menú. Quan l'usuari clica "Iniciar", el Controlador comença a coordinar la feina.

Al principi, teníem pensat crear fils d'execució separats (`Thread`) perquè el programa calculés els temps asimptòtics en segon pla mentre la finestra es dibuixava. El problema és que Java Swing és una mica delicat amb els fils; si intentes dibuixar a la pantalla mentre un altre fil està tocant les estructures de dades, el programa sol fallar, llançar excepcions o quedar-se totalment congelat. [1]

Per solucionar això d'una manera molt més neta i senzilla, hem decidit fer servir la classe `javax.swing.Timer`. Hem programat aquest temporitzador perquè faci una acció cada certs mil·lisegons. Cada vegada que es crida aquesta acció a través del Timer, el Controlador crida el Model perquè calculi només *un punt nou* de la gràfica. Un cop té aquest punt, li envia a la Vista i li demana que actualitzi el dibuix cridant al mètode `repaint()`.

Gràcies a aquest truc del `Timer`, no saturen la CPU amb un bucle infinit. El processador s'allibera constantment entre punt i punt, cosa que permet que l'animació de la gràfica es vegi completament fluïda i que l'usuari pugui prémer el botó "Aturar" o moure la finestra en qualsevol moment sense que l'aplicació deixi de respondre.

3 Problemes del Maquinari i Solucions

L'objectiu de la pràctica no és mostrar funcions matemàtiques, sinó mesurar la resposta en temps d'un maquinari quan se l'estressa. Hem descobrit que a la pràctica apareixia remou al pintar les gràfiques, per això hem intentat trobar una solució al problema. [2]

3.1 El fenomen del Jitter

Un algorisme de $O(n)$ sobre valors moderats s'executa en valor molt baixos de nanosegons en un processador modern. El temps necessari per al sistema operatiu d'es-coltar i registrar aquest cronòmetre mitjançant
`System.nanoTime()` està subjecte a tenir interferències, ja que el processador pot dónar més importància a altres operacions.Per exemple, si el sistema operatiu atén una petició de xarxa en aquest instant, els nanosegons calculats poden reflectir un pic que multipliqui per molt el cost real teòric. Això generava que la línia de la gràfica d'$O(n)$ fóssin pics alts i baixos.

La decisió de disseny que hem optat ha estat aplicar un mostreig a través d'una mitjana. En la funció pertinent del `Modelo`, si se selecciona $O(n)$, s'executa forçosament 10.000 vegades el càlcul abans de mirar el cronòmetre final, i es divideix el resultat total:

$$T_n = \frac{\sum_{k=1}^{10000}(C\text{àlcul}(n))}{10000} \tag{1}$$

Això actua com a filtre. El renou aleatori del sistema operatiu perd l'efecte i s'aconsegueixen unes línies més rectes i assimptòticament millors. Per $O(n^3)$ el valor de repetició és 1, atès que una sola passa ja ocupa prou temps perquè el renou del maquinari tengui importància.

3.2 L'escalfament de Java (Warm-up)

Java triga una mica a optimitzar el codi la primera vegada que s'executa. Perquè la gràfica no comenci amb un salt raro, feim que el programa faci un encalentiment previ fent uns càlculs en silenci al executar l'aplicació. Així, quan l'usuari clica "Iniciar", el processador ja va a tota velocitat.

La decisió presa per a combatre aquestes baixades i pujades ha estat cridar a la funció que simula les complexitats al constructor del model de dades:

```
public Modelo() {
    simular(500, "O(n)");
    simular(500, "O(n log n)");
    simular(100, "O(n^2)");
}
```

*És recomanable esperar uns segons abans d'iniciar la representació de les gràfiques.

4 Estratègies de Renderització Eficient

La representació visual de dades asimptòtiques presenta reptes de rendiment i geometria que hem hagut de resoldre mitjançant tècniques de mostreig i transformacions de coordenades.

4.1 Optimització del Càlcul per Mostreig

En lloc de realitzar el càlcul computacional per a cada valor d'N fins arribar a 5.000, hem implementat un sistema de segmentació de dades. Hem determinat que 20 punts equidistants proporcionen una resolució suficient per representar la curvatura de qualsevol funció polinòmica sense saturar la CPU. Aquest valor és modificable, com més punts millor detall de la corba.

Això implica que el Controlador només realitza una crida al Model cada 5% del recorregut total (passos de 250 unitats). Aquesta decisió redueix dràsticament el cost d'execució del programa, especialment en el cas de $O(n^3)$, ja que passem de 5.000 simulacions pesades a només 20, sense que l'usuari percebi cap pèrdua de precisió visual.

4.2 Mapeig de Coordenades

Un dels reptes més importants ha estat la transformació de les unitats matemàtiques (nanosegons i valor N) a coordenades de pantalla (píxels). Hem optat per un model de proporció relativa, que vol dir això? Significa que les gràfiques es reescalen en funció del tamany de la pantalla, o sigui del `JPanel`:

1. Calculem el percentatge que ocupa el punt respecte el màxim actual ($P_x = N/N_{max}$ i $P_y = T/T_{max}$).
2. Apliquem aquest percentatge a l'espai de la finestra, restant els marges de seguretat que tenim definits.
3. Invertim l'eix Y, ja que en `Swing` l'origen $(0, 0)$ es troba a la part superior, mentre que en una gràfica matemàtica es troba a la part inferior.

Aquest enfocament garanteix que la gràfica sigui completament modificable: si l'usuari redimensiona la finestra, les corbes es recalculen instantàniament sense deformar-se.

A continuació, les fórmules de cada eix:

$$X_{pant} = Marg + \left(\frac{N}{N_{max}}\right) \times (W_{util}) \quad (2)$$

$$Y_{pant} = (H - Marg) - \left(\frac{T}{T_{max}}\right) \times (H_{util}) \quad (3)$$

On W_{util} i H_{util} són l'amplada i altura restant després de treure els marges. Això fa que la gràfica sigui elàstica i s'adapti a la mida de la finestra.

4.3 Normalització Dinàmica i Escalat

Per evitar que les corbes quedin aixafades o surtin dels límits visuals, hem implementat un algorisme d'auto-escalat. Abans de cada repintat, el programa cerca el valor temporal màxim (T_{max}).

Aquesta normalització és molt important per la comparació: quan visualitzem només $O(n)$, el sostre de la gràfica s'ajusta a pocs milers de nanosegons, permetent veure la inclinació de la recta. En el moment que l'usuari activa $O(n^3)$, el T_{max} augmenta fins a valors de milions, i el sistema reescala automàticament totes les altres línies cap a la part inferior, reflectint així la diferència d'ordre de magnitud real entre els algorismes. Per això quan executam totes les complexitats alhora la única corba que veim ben diferenciada és la $O(n^3)$.

5 Guia d'Usuari

L'aplicació ha estat dissenyada per ser clara i funcional, facilitant la comparativa entre ordres de magnitud de forma visual i senzilla.

5.1 Interfície i Inici de la Simulació

En executar el programa, l'usuari troba una finestra principal amb un llenç central on es dibuixaràn els eixos. Per començar, cal triar el tipus d'algorisme a través del menú superior o prémer directament el botó "Iniciar Simulació" situat a la part inferior. Aquest botó activa el `Timer` que anirà demanant els punts progressivament fins arribar al límit de $N = 5.000$. A més, a la part lateral dreta observam un panell on apareixerà el temps que tarda cada algorisme.

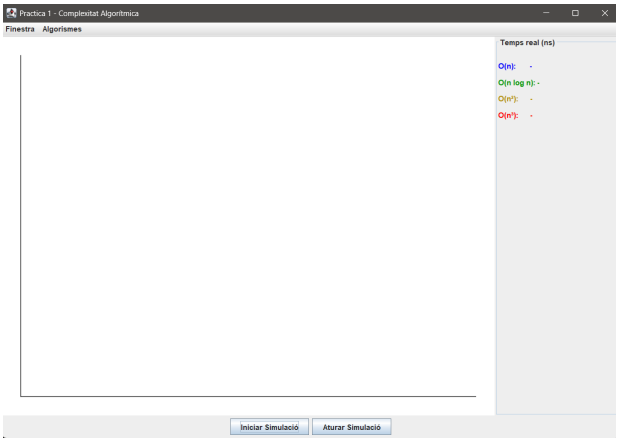

Figura 1: Interficie base

5.2 Selecció d'Algorismes

El programa permet comparar les quatre complexitats de cop mitjançant l'opció "Tots" (que és la predeterminada), o bé centrar-se en una de sola per observar-ne millor la forma gràcies a l'auto-escalat dinàmic. En qualsevol moment, l'usuari pot aturar el procés prement el botó "Aturar Simulació" per analitzar les dades recollides fins a aquell instant.

5.3 Interpretació de Resultats

A mesura que la simulació avança, el panell lateral dret mostra els nanosegons exactes de l'última execució de cada algorisme. Això permet interpretar

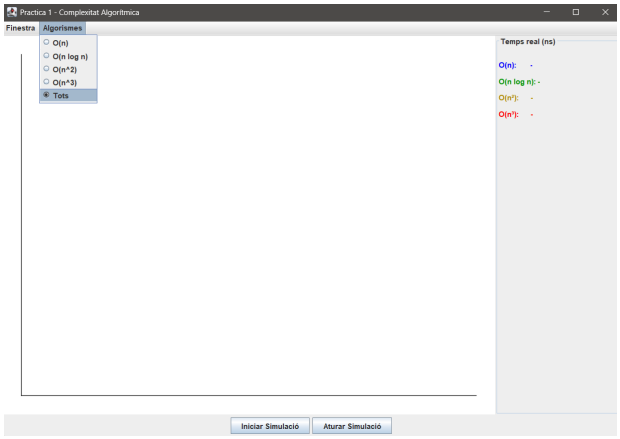

Figura 2: Selector de l'algorisme

els resultats tant des d'un punt de vista visual (mitjançant la corba de la gràfica) com numèric, observant el temps exacte que tarda cada simulació.

Quan es representen totes les complexitats simultàniament, la diferència d'ordres de magnitud fa que la corba de $O(n^3)$ domini visualment la gràfica. Per aquest motiu, també és interessant observar cada complexitat de manera individual. Gràcies al sistema d'auto-escalat implementat al panell gràfic, quan només es representa un algorisme el límit superior de la gràfica s'ajusta automàticament, permetent apreciar amb molta més claredat la forma característica de cada funció.

A continuació es mostren exemples de la representació individual de cada complexitat:

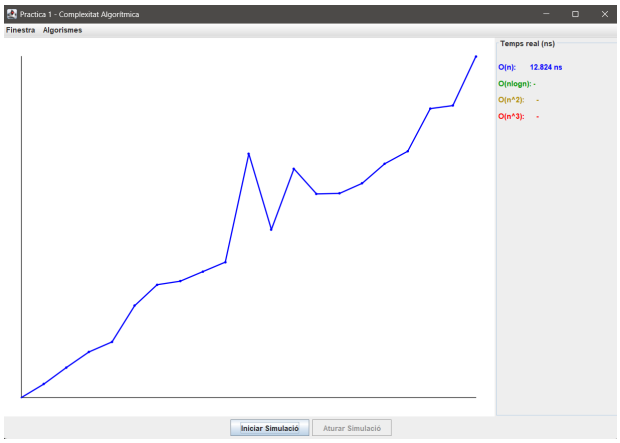

Figura 3: Representació individual de la complexitat $O(n)$

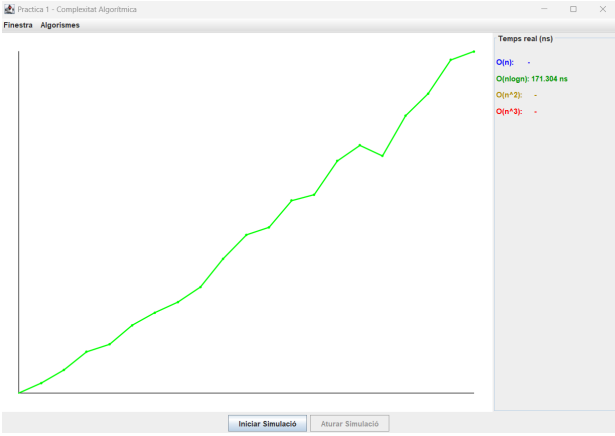

Figura 4: Representació individual de la complexitat $O(n \log n)$

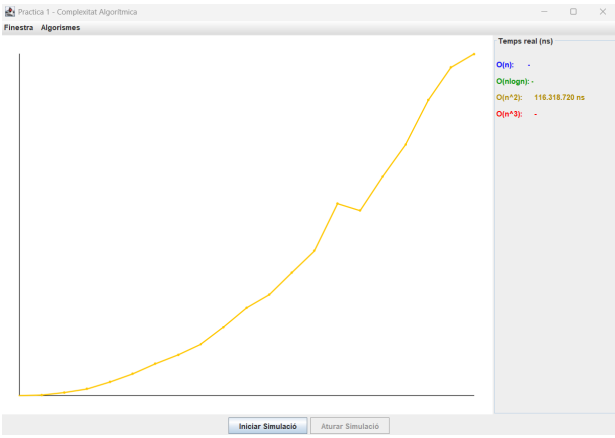

Figura 5: Representació individual de la complexitat $O(n^2)$

Aquestes representacions permeten observar clarament la diferència en el ritme de creixement entre els diferents algorismes. Mentre que $O(n)$ mostra un creixement lineal suau, $O(n \log n)$ comença a separar-se progressivament. En canvi, $O(n^2)$ presenta una curvatura parabòlica molt més pronunciada, i finalment $O(n^3)$ creix de manera extremadament ràpida, cosa que explica per què aquesta complexitat domina la visualització quan totes es representen conjuntament.

6 Conclusions

Aquesta pràctica ens ha permès diferenciar la teoria de la notació Big-O i la realitat d'execució d'un processador. Hem après que, encara que un algorisme sigui lineal ($O(n)$) a la pissarra, mesurar-ne el

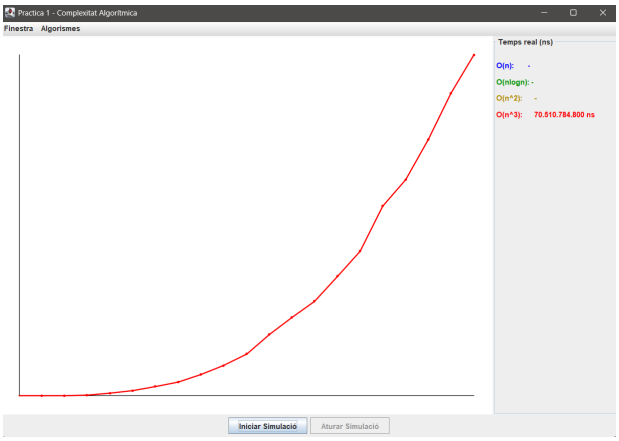

Figura 6: Representació individual de la complexitat $O(n^3)$

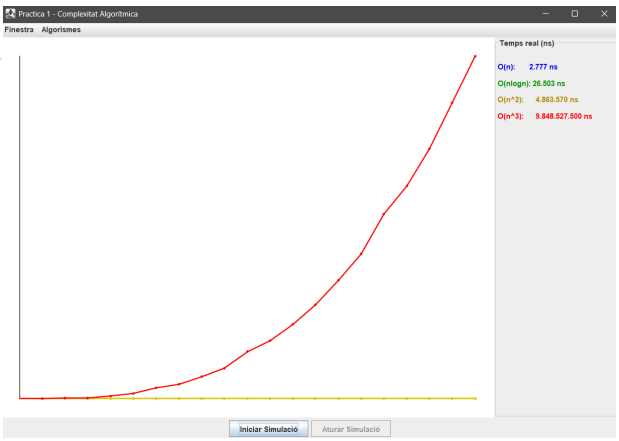

Figura 7: Representació de totes les complexitats

seu rendiment a través d'una aplicació informàtica requereix controlar moltes variables externes, com el renou del sistema operatiu o el comportament de la JVM.

L'ús d'un temporitzador (`Timer`) i la segmentació del càlcul en 20 punts han estat decisions clau per mantenir l'aplicació fluida. Aquestes optimitzacions ens han ensenyat que el rendiment no només depèn de l'algorisme de càlcul, sinó també de com gestionem la interfície gràfica i el flux de dades. Separar les responsabilitats mitjançant el patró MVC ens ha permès solucionar cada problema (càlculs, visualització i control) de forma independent i ordenada.

Finalment, la integració de tècniques com l'escalfament del codi (*warm-up*) i la normalització per percentatges del panell gràfic ens ha donat com a resultat una eina capaç de mostrar cada complexitat de forma neta i entenedora. En conclusió, la pràctica no només ens ha servit per entendre millor el cost algorísmic, sinó per aprendre a dissenyar programes amb el patró MVC i a entendre les complexitats i simpleses que ens aporta.

Referències

[1] Oracle Corporation, *Concurrencia amb Swing*, The Java Tutorials. [Online]. Available: https://docs.oracle.com/javase/tutorial/uiswing/concurrency/

[2] Google Gemini AI, *Asistència en maquetació LaTeX i millora de les gràfiques*, 2026.